Experiment 1 ? Program 6

DATA SCIENCE - 22DS102006

6. Time Series Analysis

Create time series using datetime object in pandas indexed by timestamps.

Use pandas.date_range to generate a DatetimeIndex with an indicated length.

Generate data ranges by setting time zone, localize time zone and convert to particular time zone using tz_convert and combine two different time zones.

Perform period arithmetic such as adding and subtracting integers from periods and construct range of periods using period_range function.

Convert Periods and PeriodIndex objects to another frequency with asfreq method.

Convert Series and DataFrame objects indexed by timestamps to periods with the to_period method.

Perform resampling, downsampling and upsampling for the time series.

Create time series using datetime object in pandas indexed by timestamps.

Aim

To create a time series in Pandas using **datetime** objects and use the timestamps as the index of the time series.

Syntax

```
pd.to_datetime(date_values)
```

```
pd.Series(data, index=datetime_index)
```

General syntax:

```
import pandas as pd
```

```
dates = pd.to_datetime([...])
```

```
series = pd.Series(data, index=dates)
```

Program Solution

```
import pandas as pd
```

```
# Create datetime objects
dates = pd.to_datetime([
    '2026-01-01',
    '2026-01-02',
    '2026-01-03',
    '2026-01-04',
    '2026-01-05'
])
# Create data
temperature = [28, 30, 29, 31, 32]
# Create time series with timestamps as index
time_series = pd.Series(
    temperature,
    index=dates
)
# Display the time series
print("Time Series:")
print(time_series)
```

Output

```
Time Series:
2026-01-01    28
2026-01-02    30
2026-01-03    29
2026-01-04    31
2026-01-05    32
dtype: int64
```

The index of the Series consists of timestamps, while the corresponding values represent the temperature recorded on each date.

Use pandas.date_range to generate a DatetimeIndex with an indicated length.

Aim

To generate a **DatetimeIndex** of a specified length using the pandas.date_range() function.

Syntax

```
pd.date_range(start=None, end=None, periods=None, freq='D')
```

Important Parameters

start – Starting date/time.

end – Ending date/time.

periods – Number of timestamps to generate.

freq – Frequency of timestamps, such as D (daily), h (hourly), M (monthly).

For a specified length, the commonly used syntax is:

```
pd.date_range(start='YYYY-MM-DD', periods=n, freq='D')
```

Program Solution

```
import pandas as pd
```

```
# Generate a DatetimeIndex with 7 dates
```

```
date_index = pd.date_range(
```

```
    start='2026-01-01',
```

```
    periods=7,
```

```
    freq='D'
```

```
)
```

```
# Display the DatetimeIndex
```

```
print("Generated DatetimeIndex:")
```

```
print(date_index)
```

Output

```
Generated DatetimeIndex:
```

```
DatetimeIndex(['2026-01-01', '2026-01-02', '2026-01-03',
               '2026-01-04', '2026-01-05', '2026-01-06',
               '2026-01-07'],
              dtype='datetime64[ns]', freq='D')
```

Explanation

The pd.date_range() function is used to generate a sequence of dates at regular intervals.

In the program:

```
pd.date_range(
    start='2026-01-01',
    periods=7,
    freq='D'
)
```

Step-by-step

start='2026-01-01'
Specifies the starting date.

periods=7
Specifies that exactly 7 timestamps should be generated.

freq='D'
Specifies a daily frequency, so each timestamp is one day apart.

The resulting object is a DatetimeIndex.

Use pandas.date_range to generate a DatetimeIndex with an indicated length.

Aim

To generate date ranges by setting a time zone, localizing a time zone, converting timestamps to another time zone using **tz_convert()**, and combining time-series data from two different time zones using Pandas.

Syntax

1. Generate date range with a time zone

```
pd.date_range(start, periods=n, freq='h', tz='Asia/Kolkata')
```

2. Localize a time zone

```
datetime_index.tz_localize('Asia/Kolkata')
```

3. Convert to another time zone

```
datetime_index.tz_convert('America/New_York')
```

4. Combine time series

```
pd.concat([series1, series2])
```

Program Solution

```
import pandas as pd

--------------------------------------------------

1. Generate date range by setting a time zone

--------------------------------------------------

india_dates = pd.date_range(

    start='2026-01-01 09:00',

    periods=3,

    freq='h',

    tz='Asia/Kolkata'

)

print("1. Date Range with Asia/Kolkata Time Zone:")

print(india_dates)

--------------------------------------------------

2. Create a timezone-naive DatetimeIndex

--------------------------------------------------

dates = pd.date_range(
```

```
    start='2026-01-01 09:00',
    periods=3,
    freq='h'
)
print("\n2. Timezone-naive Date Range:")
print(dates)
--------------------------------------------------
3. Localize the timezone
--------------------------------------------------
localized_dates = dates.tz_localize('Asia/Kolkata')
print("\n3. After Localizing to Asia/Kolkata:")
print(localized_dates)
--------------------------------------------------
4. Convert to another timezone using tz_convert()
--------------------------------------------------
new_york_dates = localized_dates.tz_convert(
    'America/New_York'
)
print("\n4. Converted to America/New_York:")
print(new_york_dates)
--------------------------------------------------
5. Create another time series in a different
   timezone
--------------------------------------------------
india_series = pd.Series(
    [100, 200, 300],
```

```
    index=localized_dates
)
new_york_series = pd.Series(
    [400, 500, 600],
    index=new_york_dates
)
print("\n5. India Time Series:")
print(india_series)
print("\nNew York Time Series:")
print(new_york_series)
--------------------------------------------------
6. Combine two different timezone series
--------------------------------------------------
combined_series = pd.concat([
    india_series,
    new_york_series
])
print("\n6. Combined Time Series:")
print(combined_series)
```

Sample Output

1. Date Range with Time Zone

```
1. Date Range with Asia/Kolkata Time Zone:
DatetimeIndex(['2026-01-01 09:00:00+05:30',
               '2026-01-01 10:00:00+05:30',
               '2026-01-01 11:00:00+05:30'],
              dtype='datetime64[ns, Asia/Kolkata]', freq='h')
```

2. Timezone-Naive Date Range

```
2. Timezone-naive Date Range:
```

```
DatetimeIndex(['2026-01-01 09:00:00',
               '2026-01-01 10:00:00',
               '2026-01-01 11:00:00'],
              dtype='datetime64[ns]', freq='h')
```

3. Localized Time Zone

```
3. After Localizing to Asia/Kolkata:
```

```
DatetimeIndex(['2026-01-01 09:00:00+05:30',
               '2026-01-01 10:00:00+05:30',
               '2026-01-01 11:00:00+05:30'],
              dtype='datetime64[ns, Asia/Kolkata]', freq='h')
```

4. Converted to New York Time

```
4. Converted to America/New_York:
```

```
DatetimeIndex(['2025-12-31 22:30:00-05:00',
               '2025-12-31 23:30:00-05:00',
               '2026-01-01 00:30:00-05:00'],
              dtype='datetime64[ns, America/New_York]', freq='h')
```

5. Time Series

```
5. India Time Series:
```

```
2026-01-01 09:00:00+05:30    100
```

```
2026-01-01 10:00:00+05:30    200
```

```
2026-01-01 11:00:00+05:30    300
```

```
dtype: int64
```

```
New York Time Series:
```

```
2025-12-31 22:30:00-05:00    400
```

```
2025-12-31 23:30:00-05:00    500
```

```
2026-01-01 00:30:00-05:00    600
```

```
dtype: int64
```

6. Combined Time Series

```
6. Combined Time Series:
```

```
2025-12-31 22:30:00-05:00    400
```

```
2025-12-31 23:30:00-05:00    500
```

```
2026-01-01 00:30:00-05:00    600
```

```
2026-01-01 09:00:00+05:30    100
```

```
2026-01-01 10:00:00+05:30    200
```

```
2026-01-01 11:00:00+05:30    300
```

```
dtype: int64
```

Perform period arithmetic such as adding and subtracting integers from periods and construct range of periods using period_range function.

Aim

To perform period arithmetic by adding and subtracting integers from Pandas Period objects and to construct a range of periods using the period_range() function.

Syntax

1. Create a Period

```
pd.Period('2026-01', freq='M')
```

2. Add an integer to a Period

```
period + integer
```

3. Subtract an integer from a Period

```
period - integer
```

4. Construct a range of periods

```
pd.period_range(start, periods=n, freq='M')
```

Program Solution

```
import pandas as pd

# Create a monthly Period

p = pd.Period('2026-01', freq='M')

print("Original Period:")

print(p)

# Add integers to the period

print("\nAfter adding 2:")

print(p + 2)

print("\nAfter adding 5:")

print(p + 5)

# Subtract integers from the period

print("\nAfter subtracting 1:")

print(p - 1)

print("\nAfter subtracting 3:")

print(p - 3)

# Create a range of monthly periods

periods = pd.period_range(

    start='2026-01',

    periods=6,

    freq='M'

)

print("\nRange of Periods:")

print(periods)
```

Sample Output

```
Original Period:
```

```
2026-01
```

```
After adding 2:
```

```
2026-03
```

```
After adding 5:
```

```
2026-06
```

```
After subtracting 1:
```

```
2025-12
```

```
After subtracting 3:
```

```
2025-10
```

```
Range of Periods:
```

```
PeriodIndex(['2026-01', '2026-02', '2026-03',
```

```
             '2026-04', '2026-05', '2026-06'],
```

```
            dtype='period[M]')
```

Convert Periods and PeriodIndex objects to another frequency with asfreq method.

Aim

To convert Pandas **Period** and **PeriodIndex** objects from one frequency to another using the asfreq() method.

Syntax

1. Convert a Period to another frequency

```
period.asfreq(freq, how='start')
```

2. Convert a PeriodIndex to another frequency

```
period_index.asfreq(freq, how='start')
```

Parameters

freq – The target frequency, such as 'D', 'M', 'Q', or 'Y'.

how – Specifies whether to use the start or end of the period.

'start' – Beginning of the period.

'end' – End of the period.

Program Solution

```
import pandas as pd
```

```
--------------------------------------------------
```

```
1. Create a monthly Period
```

```
--------------------------------------------------
```

```
p = pd.Period('2026-01', freq='M')
```

```
print("Original Period:")
```

```
print(p)
```

```
--------------------------------------------------
```

```
2. Convert monthly Period to daily frequency
```

```
--------------------------------------------------
```

```
daily_start = p.asfreq('D', how='start')
```

```
print("\nMonthly Period converted to Daily (Start):")
```

```
print(daily_start)
```

```
daily_end = p.asfreq('D', how='end')
```

```
print("\nMonthly Period converted to Daily (End):")
```

```
print(daily_end)
```

```
--------------------------------------------------
```

```
 3. Create a PeriodIndex
```

```
 --------------------------------------------------
```

```
period_index = pd.period_range(
```

```
    start='2026-01',
```

```
    periods=3,
```

```
    freq='M'
```

```
)

print("\nOriginal PeriodIndex:")

print(period_index)

 --------------------------------------------------

 4. Convert PeriodIndex to daily frequency

 --------------------------------------------------

daily_period_index = period_index.asfreq(

    'D',

    how='start'

)

print("\nPeriodIndex converted to Daily Frequency:")

print(daily_period_index)

 --------------------------------------------------

 5. Convert PeriodIndex to daily frequency at end

 --------------------------------------------------

daily_period_index_end = period_index.asfreq(

    'D',

    how='end'

)

print("\nPeriodIndex converted to Daily Frequency (End):")

print(daily_period_index_end)
```

Sample Output

```
Original Period:

2026-01

Monthly Period converted to Daily (Start):
```

```
2026-01-01
```

```
Monthly Period converted to Daily (End):
```

```
2026-01-31
```

```
Original PeriodIndex:
```

```
PeriodIndex(['2026-01', '2026-02', '2026-03'],
```

```
            dtype='period[M]')
```

```
PeriodIndex converted to Daily Frequency:
```

```
PeriodIndex(['2026-01-01', '2026-02-01', '2026-03-01'],
```

```
            dtype='period[D]')
```

```
PeriodIndex converted to Daily Frequency (End):
```

```
PeriodIndex(['2026-01-31', '2026-02-28', '2026-03-31'],
```

```
            dtype='period[D]')
```

Convert Series and DataFrame objects indexed by timestamps to periods with the to_period method.

Aim

To convert Series and DataFrame objects indexed by timestamps into Period and PeriodIndex objects using the Pandas to_period() method.

Syntax

1. Convert a Series to a PeriodIndex

```
series.to_period(freq)
```

2. Convert a DataFrame to a PeriodIndex

```
dataframe.to_period(freq)
```

Where:

freq specifies the desired frequency.

'D' → Daily

'M' → Monthly

'Q' → Quarterly

'Y' → Yearly

Program Solution

```
import pandas as pd

-------------------------------------------------

 1. Create a Series with timestamp index

-------------------------------------------------

dates = pd.to_datetime([

    '2026-01-10',

    '2026-02-15',

    '2026-03-20',

    '2026-04-25'

])

sales = pd.Series(

    [1000, 1500, 1800, 2200],

    index=dates

)

print("Original Series:")

print(sales)

-------------------------------------------------

 2. Convert Series from timestamps to monthly periods

-------------------------------------------------

period_series = sales.to_period('M')

print("\nSeries converted to Monthly Periods:")

print(period_series)

-------------------------------------------------

3. Create a DataFrame with timestamp index
```

```
-------------------------------------------------
df = pd.DataFrame(
    {
        'Sales': [1000, 1500, 1800, 2200],
        'Profit': [200, 300, 400, 500]
    },
    index=dates
)
print("\nOriginal DataFrame:")
print(df)
 -------------------------------------------------
 4. Convert DataFrame to monthly periods
 -------------------------------------------------
period_df = df.to_period('M')
print("\nDataFrame converted to Monthly Periods:")
print(period_df)
```

Sample Output

1. Original Series

```
Original Series:
2026-01-10    1000
2026-02-15    1500
2026-03-20    1800
2026-04-25    2200
dtype: int64
```

2. Series Converted to Monthly Periods

```
Series converted to Monthly Periods:
```

```
2026-01    1000

2026-02    1500

2026-03    1800

2026-04    2200

Freq: M, dtype: int64
```

3. Original DataFrame

```
Original DataFrame:

            Sales  Profit

2026-01-10   1000     200

2026-02-15   1500     300

2026-03-20   1800     400

2026-04-25   2200     500
```

4. DataFrame Converted to Monthly Periods

```
DataFrame converted to Monthly Periods:

         Sales  Profit

2026-01   1000     200

2026-02   1500     300

2026-03   1800     400

2026-04   2200     500

Freq: M
```

Perform resampling, downsampling and upsampling for the time series.

Aim

To perform resampling, downsampling, and upsampling operations on a time series using Pandas.

Syntax

1. Resampling

```
series.resample('frequency').aggregation_function()
```

Example:

```
series.resample('D').mean()
```

2. Downsampling

```
series.resample('larger_frequency').sum()
```

Example:

```
series.resample('6h').sum()
```

3. Upsampling

```
series.resample('smaller_frequency').asfreq()
```

or

```
series.resample('smaller_frequency').ffill()
```

Program Solution

```
import pandas as pd
```

```
 ------------------------------------------------
```

```
 1. Create an hourly time series
```

```
 ------------------------------------------------
```

```
dates = pd.date_range(
```

```
    start='2026-01-01 00:00',
```

```
    periods=12,
```

```
    freq='h'
```

```
)
```

```
values = [10, 12, 15, 14, 18, 20,
```

```
          22, 21, 25, 28, 30, 32]
```

```
time_series = pd.Series(
```

```
    values,
```

```
    index=dates
```

```
)
```

```
print("Original Hourly Time Series:")
```

```
print(time_series)
```

```
 ------------------------------------------------
```

```
 2. Resampling - calculate 3-hourly mean
```

```
 ------------------------------------------------
```

```
resampled = time_series.resample('3h').mean()
```

```
print("\nResampled Time Series - 3 Hour Mean:")
```

```
print(resampled)
```

```
 ------------------------------------------------
```

```
 3. Downsampling - Hourly to 4-hourly
```

```
 ------------------------------------------------
```

```
downsampled = time_series.resample('4h').sum()
```

```
print("\nDownsampled Time Series - 4 Hour Sum:")
```

```
print(downsampled)
```

```
 ------------------------------------------------
```

```
 4. Upsampling - Hourly to 30-minute intervals
```

```
 ------------------------------------------------
```

```
upsampled = time_series.resample('30min').asfreq()
```

```
print("\nUpsampled Time Series - 30 Minute:")
```

```
print(upsampled)
```

```
 ------------------------------------------------
```

```
 5. Upsampling with Forward Fill
```

```
 ------------------------------------------------
```

```
upsampled_ffill = time_series.resample('30min').ffill()
```

```
print("\nUpsampled Time Series using Forward Fill:")
```

```
print(upsampled_ffill)
```

Sample Output

1. Original Time Series

```
Original Hourly Time Series:
```

```
2026-01-01 00:00:00    10
```

```
2026-01-01 01:00:00    12
```

```
2026-01-01 02:00:00    15
```

```
2026-01-01 03:00:00    14
```

```
2026-01-01 04:00:00    18
```

```
2026-01-01 05:00:00    20
```

```
2026-01-01 06:00:00    22
```

```
2026-01-01 07:00:00    21
```

```
2026-01-01 08:00:00    25
```

```
2026-01-01 09:00:00    28
```

```
2026-01-01 10:00:00    30
```

```
2026-01-01 11:00:00    32
```

```
Freq: h, dtype: int64
```

2. Resampling

The hourly data is resampled into 3-hour intervals and the mean is calculated.

```
Resampled Time Series - 3 Hour Mean:
```

```
2026-01-01 00:00:00    12.333333
```

```
2026-01-01 03:00:00    17.333333
```

```
2026-01-01 06:00:00    22.666667
```

```
2026-01-01 09:00:00    30.000000
```

```
Freq: 3h, dtype: float64
```

For example:

```
(10 + 12 + 15) / 3 = 12.33
```

3. Downsampling

Hourly data is reduced to 4-hour intervals and the values are summed.

```
Downsampled Time Series - 4 Hour Sum:
```

```
2026-01-01 00:00:00    51
```

```
2026-01-01 04:00:00    81
```

```
2026-01-01 08:00:00    115
```

```
Freq: 4h, dtype: int64
```

For example:

```
10 + 12 + 15 + 14 = 51
```

4. Upsampling

Hourly data is increased to 30-minute intervals.

```
Upsampled Time Series - 30 Minute:
```

```
2026-01-01 00:00:00    10.0
```

```
2026-01-01 00:30:00     NaN
```

```
2026-01-01 01:00:00    12.0
```

```
2026-01-01 01:30:00     NaN
```

```
2026-01-01 02:00:00    15.0
```

```
2026-01-01 02:30:00     NaN
```

```
2026-01-01 03:00:00    14.0
```

```
...
```

```
Freq: 30min, dtype: float64
```

The newly created 30-minute timestamps contain NaN because no original observation exists at those times.

5. Upsampling Using Forward Fill

The missing values can be filled using ffill():

```
Upsampled Time Series using Forward Fill:
```

```
2026-01-01 00:00:00    10
2026-01-01 00:30:00    10
2026-01-01 01:00:00    12
2026-01-01 01:30:00    12
2026-01-01 02:00:00    15
2026-01-01 02:30:00    15
2026-01-01 03:00:00    14
...
Freq: 30min, dtype: int64
```

My Programs :

Time Series Analysis

a. Create Time Series

```
rng = pd.date_range('2023-01-01', periods=6, freq='D')
ts = pd.Series(np.random.randn(len(rng)), index=rng)
print(ts)
```

b. Time Zone Conversion

```
ts_utc = ts.tz_localize('UTC')
ts_ind = ts_utc.tz_convert('Asia/Kolkata')
print(ts_ind)
```

c. Period Arithmetic and Conversion

```
p = pd.Period('2024Q1')
print(p + 1)
print(p.asfreq('M', 'end'))
```

d. Resampling

```
ts = pd.Series(np.random.randn(100),
index=pd.date_range('2024-01-01', periods=100))
```

```
ts_monthly = ts.resample('M').mean()
print(ts_monthly)
```